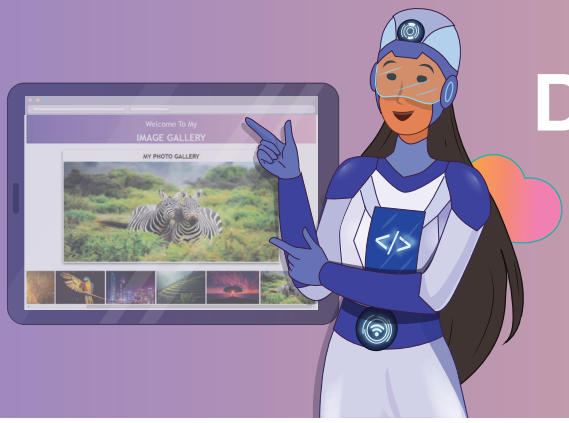


Dynamic Web Elements

05

I have a friend who is a wildlife photographer. He is confused about his portfolio design. I suggested the addition of an image gallery.

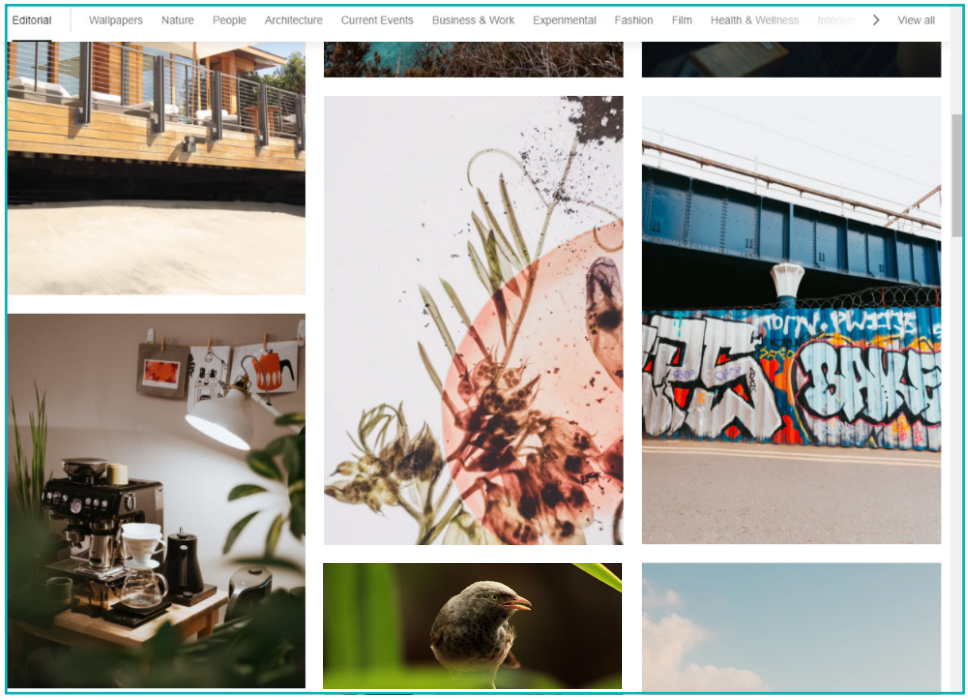
When we look at a website, what is the first thing that catches our eye? Images! A website with attractive and well-chosen images have enormous potential to engage customers. Images can be used to illustrate stories or represent products or services on a website.

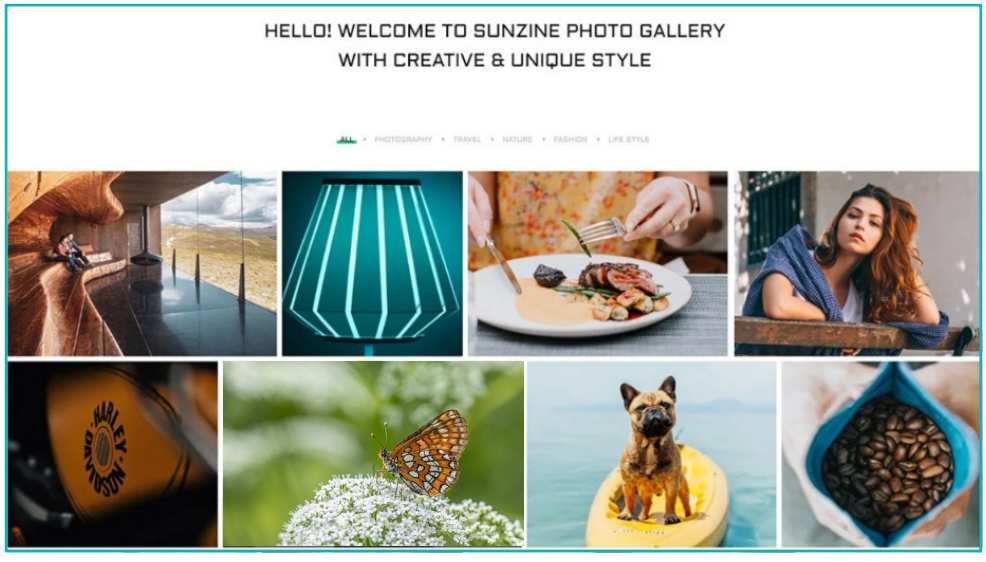
5. Dynamic Web Elements

Some websites are full of images, while some only have a few. In some cases, they are stored in an image gallery. Online shopping sites, portfolio sites and social media sites have extensive image galleries since they visually showcase their offerings.

JavaScript can be used to dynamically work with images. In this lesson, we will create an image gallery with an image preview section.

Examples of image galleries





Activity

Create a simple photography website with an image gallery. Build the structure and design using HTML and CSS.

1 Create the webpage

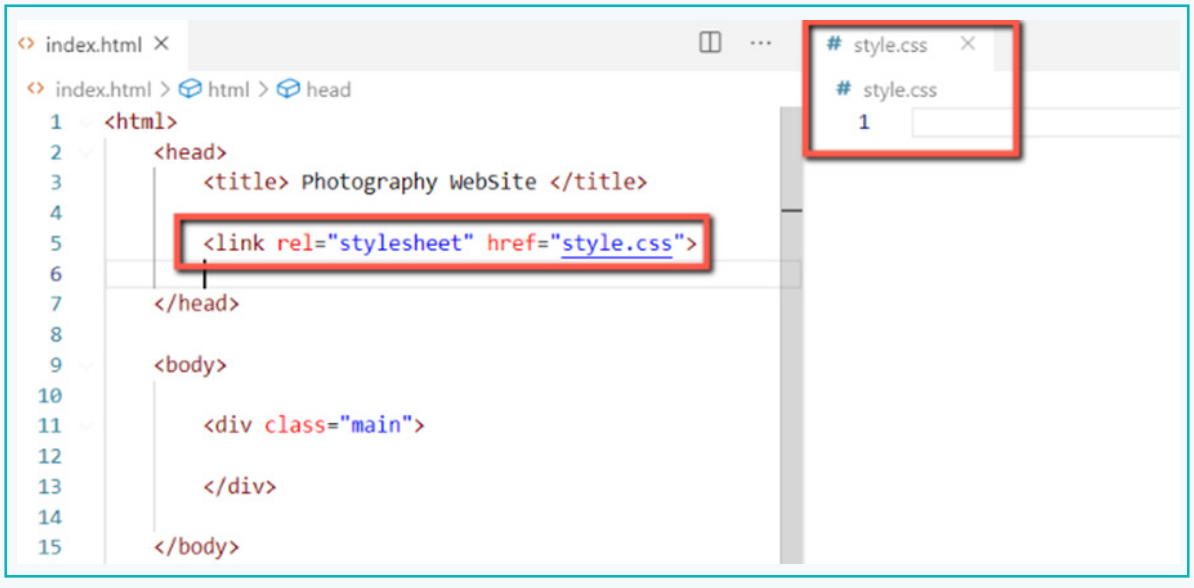
a. Open VS Code and create a new file **index.html**. Add a title tag and save it in the desired location.

```
<html>
    <head>
        <title> Photography WebSite </title>
    </head>

    <body>

    </body>
</html>
```

b. Create a div container with a class name "main". This will act as the main container inside which we will write all our HTML contents.

```
<body>
    <div class="main">

    </div>
</body>
```

c. To apply CSS styles, create a file **style.css** and link it to the website using the **link** tag.



Activity

d. Set CSS properties for the main container from the style.css file. They will give our container a fixed width and height.

e. **margin: auto** will centre align the container to the page. Set a default font for the entire application.

```
<body>
    <div class="main">

    </div>
</body>
```

```
.main {
    height: 1000px;
    width: 75%;
    margin: auto;
    font-family:'Trebuchet MS';
}
```

f. Create another div container with a class **"header"** inside the main container. Add 2 heading tags to add heading text to the website.

Note:
 tag makes the text appear bold.

```
<div class="header">
    <h2> Welcome To My </h2>
    <h1> <strong> IMAGE GALLERY </strong> </h1>
</div>
```

Welcome To My
IMAGE GALLERY

g. Design the header section by adding the displayed CSS properties.

```
.header {
    height: 200px;
    background: linear-gradient(to right, #654ea3, #eaafc8);
    color: white;
    text-align: center;
    border-radius: 5px;
}
```

Welcome To My
IMAGE GALLERY

h. Add some more CSS design properties to the heading texts.

```
.header h2 {
    padding-top: 25px;
    font-size: 40px;
    font-weight: lighter;
}
.header h1 {
    padding-top: 20px;
    font-size: 50px;
}
```

Welcome To My
IMAGE GALLERY

Activity

2 Create the image preview container

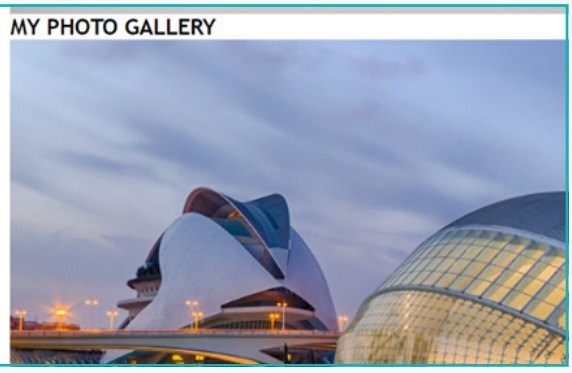
The image preview container provides a preview of the selected image. Initially, we will display a single image in this container. We will update this later using JavaScript.

a. Create a container with a class “preview_container”.

b. Provide a heading and add an image using the **<img>** tag. The **alt** attribute displays the value written inside if the image cannot be displayed.

```
<div class="preview_container">
    <h1> MY PHOTO GALLERY </h1>
    <img src="./Images/architecture.jpg" alt="No Image Selected">
</div>
```

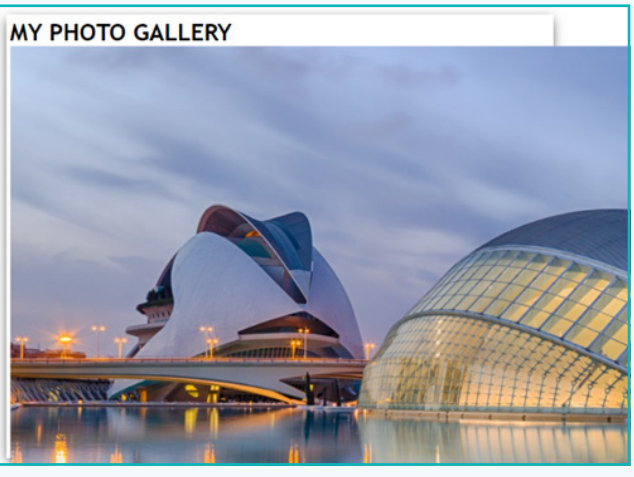
```
</div>
```



Now, we need to arrange the image and the heading inside the **preview_container** div.

c. Set the following CSS properties for this container.

```
.preview_container {
    margin-top: 20px;
    height: 600px;
    padding: 5px;
    box-shadow: 0px 5px 15px -2px rgba(0,0,0,0.75);
}
```



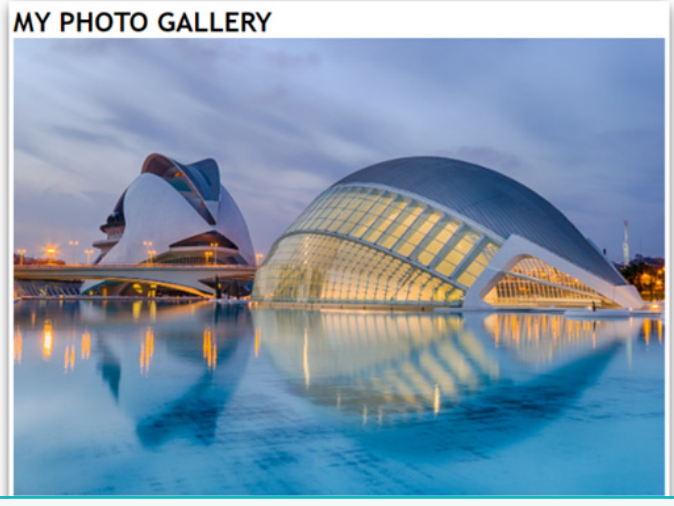
Activity

Note:

Right now, the image isn't fitting inside the container. We can fix this by providing a width and height to the image within the **preview_container** class.

Here, we have specified that the image should occupy 100% of the container's width and 90% of its height.

```
.preview_container img {
    width: 100%;
    height: 90%;
}
```



d. Finally, centre align the heading as well.

```
.preview_container h1 {
    text-align: center;
    padding-top: 10px;
    padding-bottom: 10px;
    margin: auto;
}
```

Activity

3 Create the image gallery

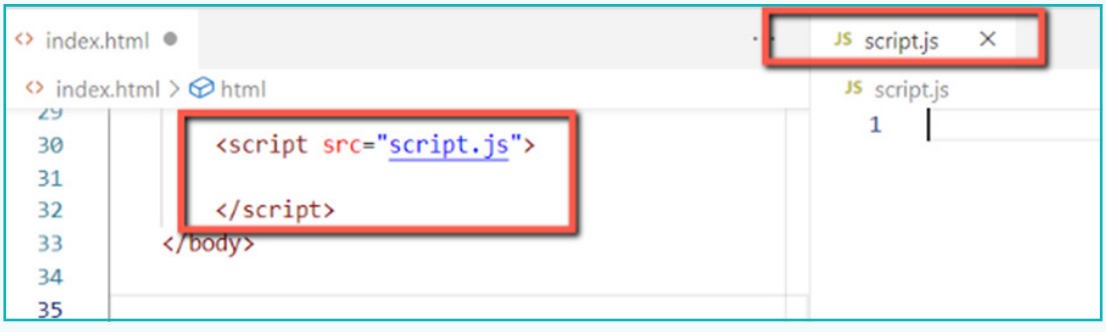
a. Create a container with a class **"gallery_container".** This container will house the image gallery.

Note:
Right now, the container is empty. We will dynamically provide its contents using JavaScript.

```
<div class="preview_container">
    <h1> MY PHOTO GALLERY </h1>
    <img src="./Images/architecture.jpg" alt="No Image Selected">
</div>

<div class="gallery_container">

</div>
```

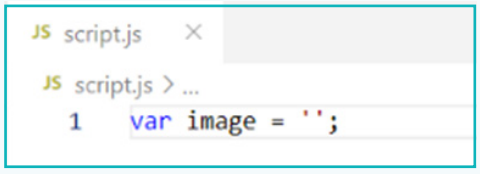
4 Implement the image gallery using JavaScript

a. Create a file **script.js** and include it in the index.html file using the **<script>** tag.



Activity

b. In the script.js file, create a variable **image** using the **var** keyword and initialize it with an empty space.



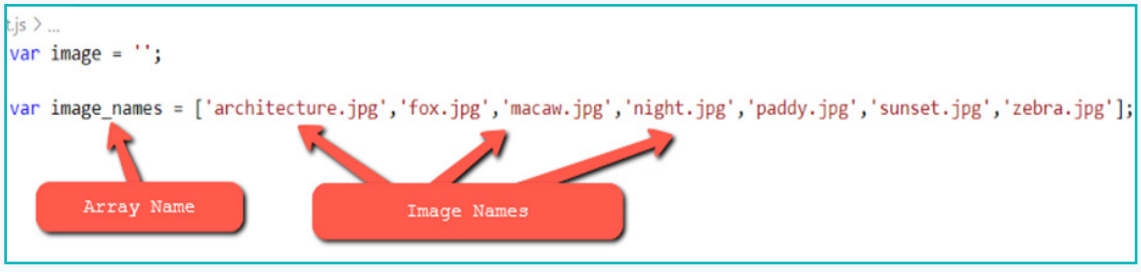
We need to store the names of all the images that we want to display in the image gallery. To store multiple values, we require an Array.

Arrays in JavaScript

Arrays are similar to variables, but arrays can store multiple values.

Arrays are created using **square brackets [ ]** and all the items within an array are stored within single or double quotes. Values in an array are separated by a **comma.**

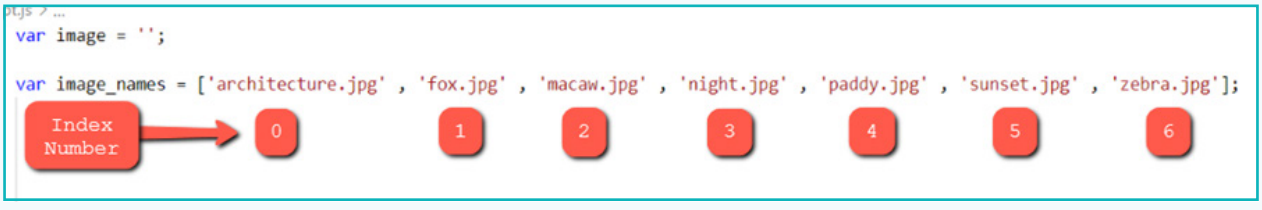
c. Create an array 'image_names' and store all the images in it.



We now need to access these names from the array and use them to display our image gallery.

Array index

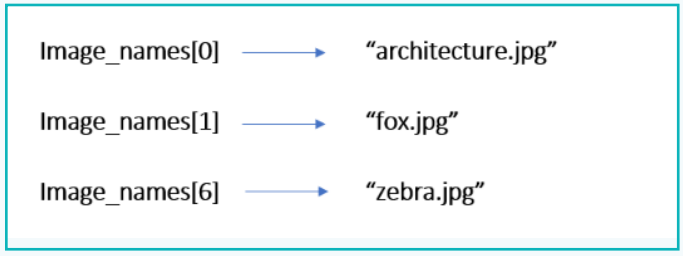
- Each item inside an array has an index number.
- The index numbers start from 0.
- Our array has 7 items, so the index number of the last item will be 6.



Activity

Accessing array items

We can access an array item using its index number. We use square brackets with the index number inside.



Now that we know how to access array items, we can use them to display our images.

d. Create a div container in the script.js file and add an img tag within it.

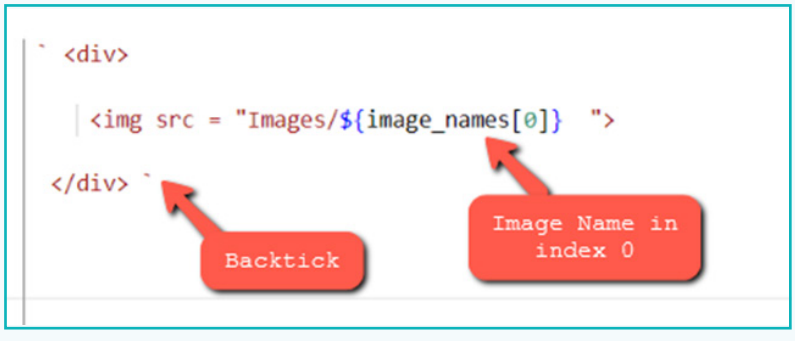
```
var image = '';

var image_names = ['architecture.jpg' , 'fox.jpg' , 'macaw.jpg' ,

<div>
    <img src = " ">
</div>
```

Notice that it displays an error. That is because we tried to write HTML code inside a JavaScript file, which does not work. We will use a different JavaScript concept called **Template Literals** to work around this.

Template literals

- Template literals are a new feature in JavaScript that allow us to write HTML contents within JavaScript code.
- They are writtten within a pair of backtick (`) symbols.
- The **${ variable }** symbol is used to embed expressions or variable values.

We will enclose HTML content within a pair of backtick (`) symbols and use ${ } to insert image names from the array.



Activity

e. Repeat the entire process and create a container for the rest of the array names as well.

```
<div>
    <img src="Images/${image_names[1]}" >
</div>

<div>
    <img src="Images/${image_names[2]}" >
</div>

<div>
    <img src="Images/${image_names[3]}" >
</div>

<div>
    <img src="Images/${image_names[4]}" >
</div>

<div>
    <img src="Images/${image_names[5]}" >
</div>

<div>
    <img src="Images/${image_names[6]}" >
</div>

`;
```

With this, we have created all the image containers for our image gallery. We need to insert all of them in the index.html file. Until then, we won't be able to view them.

Activity

Remember the image variable that we created? Let's store the containers in the image variable.

```
var image = '';
var image_names = ['architecture.jpg' , 'fox.jpg' , 'macaw.jpg' , 'night.jpg' ,
image = `<div>
            <img src="Images/${image_names[0]}" >
        </div>

        <div>
            <img src="Images/${image_names[1]}" >
        </div>

        <div>
            <img src="Images/${image_names[2]}" >
        </div>

        <div>
            <img src="Images/${image_names[3]}" >
        </div>

        <div>
            <img src="Images/${image_names[4]}" >
        </div>

        <div>
            <img src="Images/${image_names[5]}" >
        </div>

        <div>
            <img src="Images/${image_names[6]}" >
        </div>  `;
```

Now we will add this variable containing the images on the already created div container with class "gallery_container".

f. Give this container an ID "gallery_image". The purpose of assigning an ID to a container is to give it a unique identifier, which can then be accessed using JavaScript.

g. Open **index.html** file and add the ID as per the image.

```
<div class="preview_container">
    <h1> MY PHOTO GALLERY </h1>
    <img src="./Images/night.jpg" alt="No Image Selected">
</div>

<div class="gallery_container" id="gallery_image" >

</div>
```

Activity

h. Go back to the script.js file and access this container using its ID. Finally, insert the image variable in the index.html page using the innerHTML property.

View the index.html file on the browser. It should display all seven images.

```
image = `<div>
            <img src="Images/${image_names[0]}" >
        </div>

        <div>
            <img src="Images/${image_names[1]}" >
        </div>

        <div>
            <img src="Images/${image_names[2]}" >
        </div>

        <div>
            <img src="Images/${image_names[3]}" >
        </div>

        <div>
            <img src="Images/${image_names[4]}" >
        </div>

        <div>
            <img src="Images/${image_names[5]}" >
        </div>

        <div>
            <img src="Images/${image_names[6]}" >
        </div>  `;

document.getElementById('gallery_image').innerHTML = image;
```

Activity

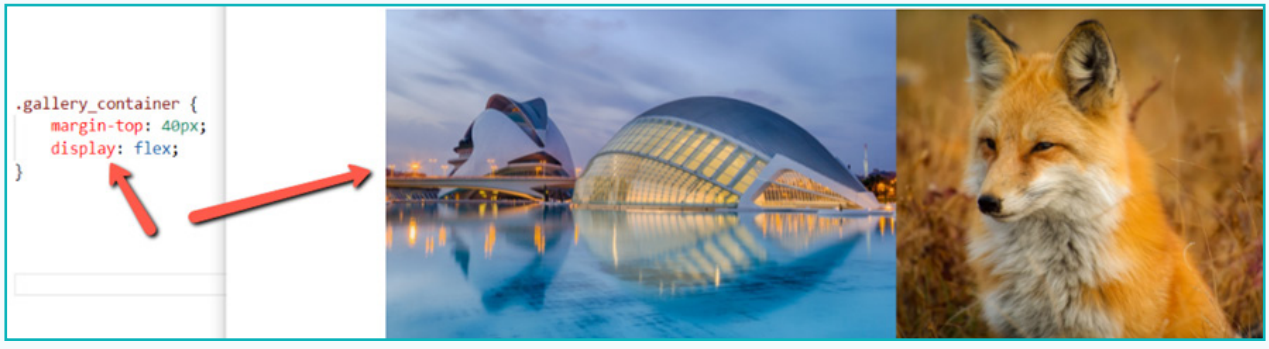
5 Design the gallery

Right now, the images are displayed vertically in the gallery. They are also displayed in their original dimensions. Both of these aspects need to be changed.

a. To display the images horizontally, convert the gallery container into a flex box.

```
.gallery_container {
    margin-top: 40px;
    display: flex;
}
```

The images are now aligned horizontally.



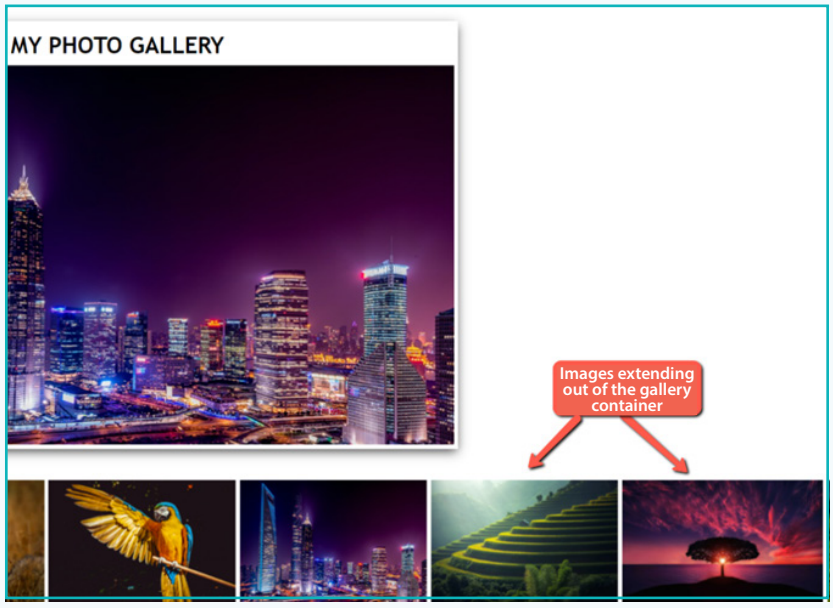
Now that our images are aligned horizontally, we need to adjust their dimensions.

b. Access the images using **.gallery_container img** and set a height and width, along with some padding.

```
.gallery_container img {
    padding: 3px;
    min-width: 20%;
    height: 180px;
}
```

Activity

The images are spilling out of the container, which we need to fix.



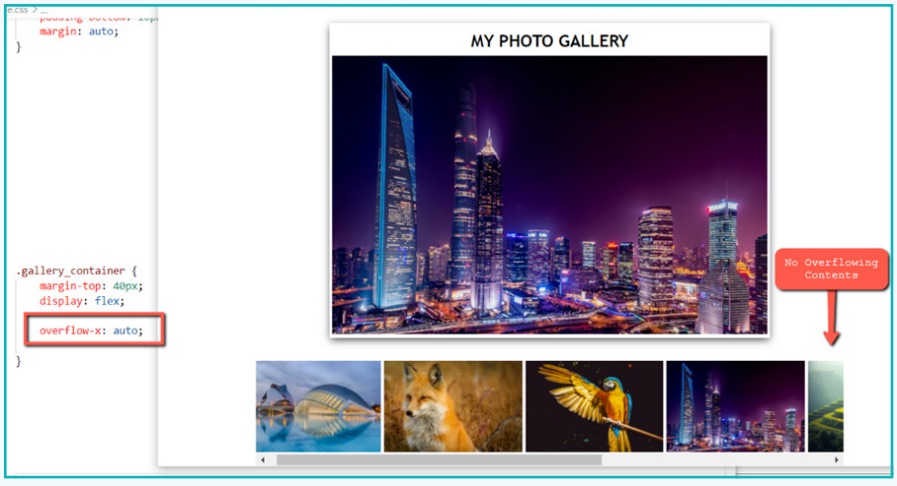
To solve this issue, we will use the overflow property in CSS.

Overflow in CSS

The overflow property specifies what should happen if the content overflows (spills outside) of a container. We can either choose to hide the overflowing contents or display a scrollbar.

For the gallery, we will display a **scrollbar.** This enables users to scroll through all the images.

c. In the **gallery_container class,** set the **overflow-x property** to **auto**. 'overflow-x:auto' creates a horizontal scrollbar along the left-right edges.



The image gallery is ready! Scroll left and right to view the images.

Activity

6 Dynamically update the preview container with the selected image

a. Open index.html file and assign an ID – **"preview_image"** to the **<img> tag** inside the **preview_container** class.

```
<div class="preview_container">

    <h1> MY PHOTO GALLERY </h1>

    <img src="./Images/night.jpg" alt="No Image Selected" id="preview_image">

</div>
```

To preview a particular image from our gallery, we need to obtain the 'src' attribute of the selected image.

b. Open the script.js file and add the 'onclick' handler to all the 7 images.

```
image = `<div>
            <img src="Images/${image_names[0]}" onclick=previewImages(this.src) >
         </div>

        <div>
            <img src="Images/${image_names[1]}" onclick=previewImages(this.src) >
        </div>

        <div>
            <img src="Images/${image_names[2]}" onclick=previewImages(this.src)>
        </div>

        <div>
            <img src="Images/${image_names[3]}" onclick=previewImages(this.src)>
        </div>
```

Note:

- onclick() is an event handler.
- previewImages() is the function where we write code to dynamically embed the selected image on the preview container.
- this.src is a pre-defined method which obtains the src attribute of the selected image.

Activity

c. Finally, define the previewImages() function.

```
document.getElementById('gallery_image').innerHTML = image;

function previewImages(image_data) {

}
```

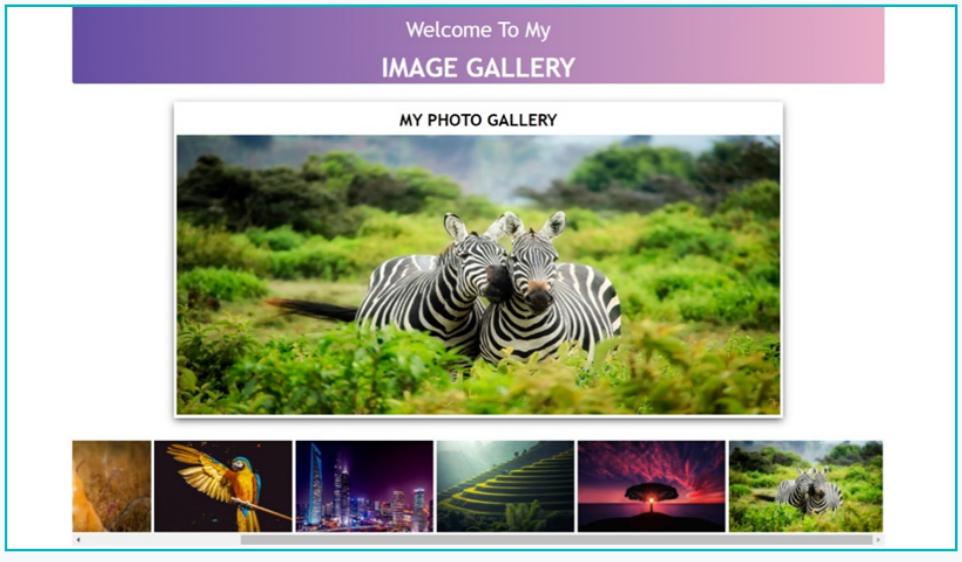
Contains src attibute's value

d. Inside this function, write the following code:

```
function previewImages(image_data) {
    document.getElementById('preview_image').setAttribute('src', image_data);
}
```

e. Run the file on the browser. Check if you can preview the selected image.

Final Output



Exercise

Fill in the blanks

1. Setting the margin property to _____ will align the contents to the centre of the page.
2. The _____HTML tag makes the font appear bold.
3. The _____ HTML attribute is used to display a text in case the image cannot be displayed.
4. To store multiple values using JavaScript, we use an _____.
5. Each array element has an index number which starts from _____.
6. To use template literals, we use the _____ symbol.
7. To insert a JavaScript variable into an HTML page, the _____ property is used.
8. The _____ CSS property is used to handle content that spills out of its container.

We started this lesson by learning about the importance of images in websites. They make a website more attractive and engage users. Then, we made our own image gallery in JavaScript using HTML and CSS. Our final gallery enables us to preview a selected image and scroll through the rest of the images.

5. Dynamic Web Elements

Static elements such as texts and images can be represented in interesting ways using these features.

Tech Flash

- **<script> tag** : Used either to write JavaScript code within an HTML page or to link to an external JavaScript file.
- **Arrays** : A data structure in JavaScript used to store multiple values. It can be created by using [ ] (square brackets) and writing the values inside the square brackets.
- **Template literals** : A new concept in JavaScript which allows us to embed variables within a string.